

Software-Hardware Co-design for Fast and Scalable Training of Deep Learning Recommendation Models

*

Dheevatsa Mudigere^{‡‡}, Yuchen Hao^{‡‡}, Jianyu Huang^{‡‡}, Zhihao Jia[§], Andrew Tulloch[‡],
Srinivas Sridharan[‡], Xing Liu[‡], Mustafa Ozdal[‡], Jade Nie[‡], Jongsoo Park[‡], Liang Luo[‡], Jie (Amy) Yang[‡], Leon Gao[‡],
Dmytro Ivchenko[‡], Aarti Basant[‡], Yuxi Hu[‡], Jiyan Yang[‡], Ehsan K. Ardestani[‡], Xiaodong Wang[‡], Rakesh Komuravelli[‡],
Ching-Hsiang Chu[‡], Serhat Yilmaz[‡], Huayu Li[‡], Jiyan Qian[‡], Zhuobo Feng[‡], Yinbin Ma[‡], Junjie Yang[‡], Ellie Wen[‡],
Hong Li[‡], Lin Yang[‡], Chonglin Sun[‡], Whitney Zhao[‡], Dmitry Melts[‡], Krishna Dhulipala[‡], KR Kishore[‡], Tyler Graf[‡],
Assaf Eisenman[‡], Kiran Kumar Matam[‡], Adi Gangidi[‡], Guoqiang Jerry Chen[‡], Manoj Krishnan[‡], Avinash Nayak[‡],
Krishnakumar Nair[‡], Bharath Muthiah[‡], Mahmoud khorashadi[‡], Pallab Bhattacharya[‡], Petr Lapukhov[‡], Maxim
Naumov[‡], Ajit Mathews[‡], Lin Qiao[‡], Mikhail Smelyanskiy[‡], Bill Jia[‡], Vijay Rao[‡]

[‡]Meta Platforms, [§]Carnegie Mellon University

ABSTRACT

Deep learning recommendation models (DLRMs) have been used across many business-critical services at Meta and are the single largest AI application in terms of infrastructure demand in its data-centers. In this paper, we present Neo, a software-hardware co-designed system for high-performance distributed training of large-scale DLRMs. Neo employs a novel 4D parallelism strategy that combines table-wise, row-wise, column-wise, and data parallelism for training massive embedding operators in DLRMs. In addition, Neo enables extremely high-performance and memory-efficient embedding computations using a variety of critical systems optimizations, including hybrid kernel fusion, software-managed caching, and quality-preserving compression. Finally, Neo is paired with *ZionEX*, a new hardware platform co-designed with Neo's 4D parallelism for optimizing communications for large-scale DLRM training. Our evaluation on 128 GPUs using 16 *ZionEX* nodes shows that Neo outperforms existing systems by up to 40× for training 12-trillion-parameter DLRM models deployed in production.

ACM Reference Format:

D. Mudigere, Y. Hao, J. Huang, and Z. Jia et al.. 2022. Software-Hardware Co-design for Fast and Scalable Training of Deep Learning Recommendation Models: . In *The 49th Annual International Symposium on Computer Architecture (ISCA '22)*, June 18–22, 2022, New York, NY, USA. ACM, New York, NY, USA, 19 pages. <https://doi.org/10.1145/3470496.3533727>

*This paper is part of the Industry Track of ISCA 2022's program.

[‡]These authors contributed equally.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ISCA '22, June 18–22, 2022, New York, NY, USA

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-8610-4/22/06...\$15.00

<https://doi.org/10.1145/3470496.3533727>

1 INTRODUCTION

Deep learning recommendation models (DLRMs) are ubiquitously used by online companies, including Amazon for selecting items in its catalog [35, 37, 58], Netflix for showing movie options [13, 29], and Google for displaying personalized advertisements [7, 9, 19].

They have also been adopted by standard benchmarking organizations, such as MLCommons (MLPerf) [38, 52]. At Meta, we have been using recommendation models extensively for ranking and click through rate (CTR) prediction, including news feed and search services [15, 17, 42, 47]. DLRMs are the single largest AI application in terms of infrastructure demand in data centers.

Unlike conventional deep neural networks (DNNs) with mainly compute-intensive operators (e.g., convolution and matrix multiplication), DLRMs combine compute-intensive components with up to thousands of data-intensive *embedding operators*, each with a different resource requirement and performance characteristic [43]. As a result, DLRMs generally exhibit much lower arithmetic intensity and larger model sizes compared to their computer vision [8, 18, 59], natural language processing [5, 10, 61], and reinforcement learning counterparts [55, 56], with models having trillions of parameters being deployed in practice, as shown in Figure 1.

Existing software and hardware solutions tailored for DNNs achieve only suboptimal performance and limited scalability on DLRMs due to the following software/hardware limitations.

On the software side, existing deep learning frameworks parallelize DNN training typically using either *data*, *model* or *pipeline parallelism* [3, 32, 48]. Frameworks that support combinations of these strategies are generally designed for specific DNN applications [16, 22, 41, 50]. However, existing parallelization strategies designed and optimized for compute-intensive DNN models achieve limited performance and scalability for DLRMs. In particular, data parallelism requires each device to save a replica of the entire model and therefore does not support DLRMs with up to trillions of parameters [32]. Moreover, a DLRM cannot be directly parallelized using model or pipeline parallelism due to the data-dependent behavior of its embedding operators. Specifically, processing different training samples may require accesses to different embedding parameters depending on the categorical inputs of each sample. This

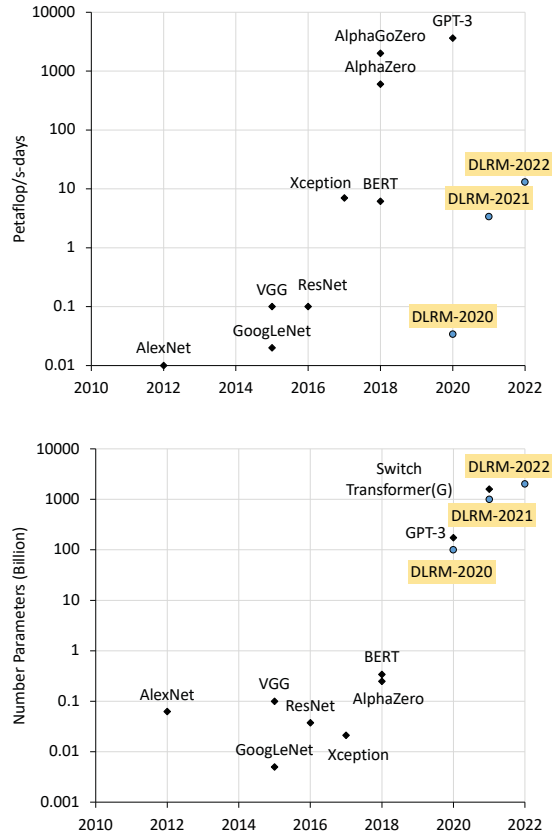


Figure 1: Comparing deep learning models in total amount of compute, in petaflop/s-days (top) [45] and model capacity (bottom).

data-dependent behavior makes it infeasible to statically partition a DLRM’s trainable parameters into disjoint subsets while satisfying data dependencies for all samples, a necessity for using model and pipeline parallelism.

In addition, today’s DNN frameworks are designed and optimized for compute-intensive DNN computations and miss critical optimizations for data-intensive embedding operators. Specifically, DLRMs contain up to thousands of embedding operators. The forward processing, backward propagation, and gradient synchronization for these embedding operators require launching thousands of CUDA kernels in a training iteration and consume up to terabytes of aggregated GPU device memory, introducing significant runtime overheads and memory requirements.

On the hardware side, modern hardware platforms such as GPU-based clusters provide significant capability boost, but they are not designed to match the performance characteristics of DLRMs. Specifically, hardware platforms for DNN training are generally optimized for centralized inter-node communications (e.g., parameter servers [3]) and/or AllReduce communications (e.g., Horovod [54] and NCCL [1]). However, as identified in Section 3, performant and scalable DLRM training requires efficient hardware support for a

mixture of diverse communication patterns, including AllReduce, AlltoAll, ReduceScatter, OneToMany, and ManyToMany.

1.1 Our Approach

We present Neo, a software-hardware co-designed system for fast and scalable DLRM training building on top of three key techniques.

4D parallelism. To enable fast and scalable training of the massive embedding operators in DLRMs, it is crucial to effectively balance the workload distribution across GPUs while minimizing communication costs. We introduce a *4D parallelism* strategy that combines *table-wise*, *row-wise*, *column-wise*, and *data* parallelism to jointly optimize the parallelization performance of embedding operators. Additionally, Neo also supports applying 4D parallelism in a *recursive* manner at different levels of hardware hierarchy to further improve load balance and hardware efficiency.

High-performance embedding computation. Neo employs two novel optimizations to minimize the computational costs and memory requirements of embedding operators. First, we introduce a *hybrid kernel fusion* technique that fuses (1) multiple embedding operators and (2) embedding computations and their parameter updates *all* in a single CUDA kernel. This is realized by co-designing the optimization algorithms and software implementation of embedding operators. Second, to provide sufficient memory capacity for DLRM training, Neo uses a *software-managed caching* mechanism to leverage the memory hierarchy of modern hardware platforms. Finally, a variety of compression techniques [29, 63] are further applied to minimize memory requirements.

Hardware platform design. We introduce *ZionEX*, a new hardware platform co-designed with Neo’s 4D parallelism to optimize inter-node communications for distributed DLRM training. *ZionEX* supports a *fully-connected* topology across all GPUs in the cluster by using a dedicated *RDMA over Converged Ethernet* (RoCE) based scale-out network. This topology design promotes high-performance data transfers for the performance-dominating communication workloads (e.g., AlltoAll and ManyToMany) in distributed DLRM training. Meanwhile, *ZionEX* supports both the RDMA and GPUDirect communication protocols and retains flexible intra-node GPU fabric. This enables high-performance DLRM training on *ZionEX*, while ensuring compatibility with existing data-center infrastructure to allow wide deployment of *ZionEX*.

Results. We have evaluated Neo on three DLRMs deployed in production for different tasks, including click through rate prediction, ranking, and engagement, representing a diverse set of production-level recommendation models. Our evaluation on 128 A100 GPUs on 16 *ZionEX* nodes shows that Neo is able to process up to 1.7 million queries per second for training DLRMs with 12 trillion parameters, a 40× speedup compared to existing solutions for DLRM training in production. Ablation studies show that 4D parallelism, high-performance embedding computation, and the new *ZionEX* platform are all critical to enabling fast and scalable DLRM training.

Table 1: Sample DLRM time to train latency resources demand

Total compute	1+ PF/s
Total memory capacity	1+ TB
Total memory BW	100+ TB/s
Network injection BW per worker	100+ GB/s
Network bisection BW	1+ TB/s

To summarize, our contributions are:

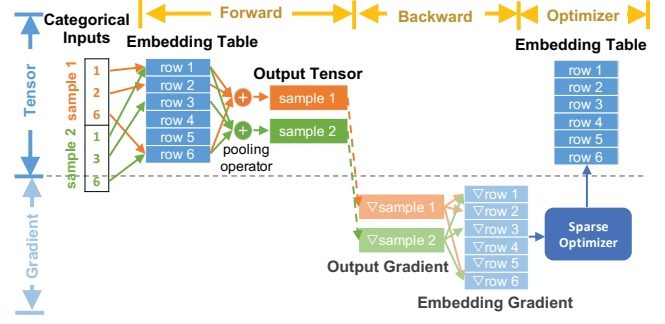
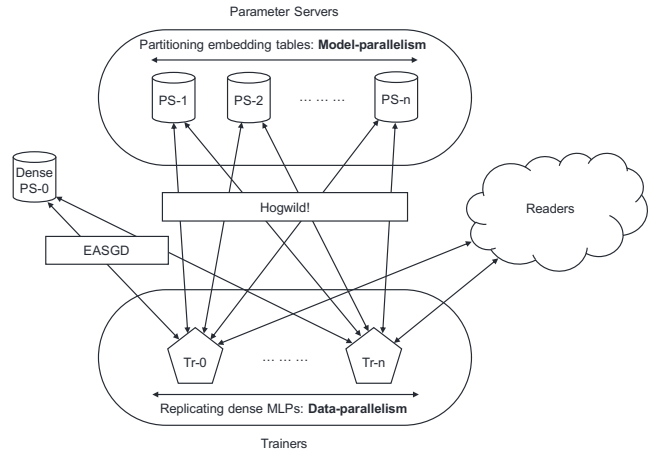
- We present Neo, a software-hardware co-designed system for fast and scalable training of DLRMs. Neo outperforms existing systems by up to 40× for training large-scale DLRMs with 12 trillion parameters.
- We propose 4D parallelism, a combination of table-wise, row-wise, column-wise, and data parallelism for training embedding operators.
- We develop and implement high-performance embedding operators using hybrid kernel fusion, software-managed caching, and quality-preserving compression.
- We build *ZionEX*, a new hardware platform co-designed with Neo’s 4D parallelism to accelerate a variety of communication patterns in DLRM training.

2 BACKGROUND

DLRMs typically have two modes of training - offline and online, each with varying requirements. The offline training can be viewed more as a pre-training, where a candidate model is trained on sufficiently large historical data, and expected to generalize when deployed to current/unseen samples. Once deployed, DLRMs continue to be trained in an online mode using the data it has already served on. Offline training is throughput limited, fitting into the more conventional “train as fast as possible on as much data as possible” paradigm, whereas online training is more latency sensitive, with the frequency of re-training and update being an important factor. For online training, the throughput requirement is lower hence it might be desired to use proportionally lower resources. This creates a unique requirement of training very large models at smaller scales capable of tolerating lower throughput.

This paper focuses on offline training with more demanding training throughput needs — up to millions of samples (queries) per second resulting from processing through tens of petabytes of training data within a reasonable time. This drives the training platform requirements, as summarized in Table 1.

Embedding operators. A major difference between DLRMs and conventional deep neural networks is leveraging categorical features such as users, posts, or pages. The DLRMs used in production typically contain up to thousands of categorical features, each of which corresponds to a dedicated *embedding operator*. An embedding operator takes as an input a multi-hot vector, and each non-zero element in the vector triggers a full row retrieval in the embedding table where each index in the input vector corresponds to a table row. Finally, all embedding rows for a given input vector are combined with element-wise pooling, as shown in Fig. 2.

**Figure 2: Workflow of an embedding operator.****Figure 3: Disaggregated parameter-server based system**

Parallelization strategies. Traditionally a disaggregated parameter-server (PS) based distributed CPU training system has been used for training DLRMs in a production setting [17, 42]. Specifically, the dense parameters from the MLP modules are duplicated between the trainers to exploit data-parallelism. Their weights are synchronized with a centralized dense parameter server using Elastic Averaging method SGD [68, 71]. On the other hand, The parameters from the embedding tables are partitioned and placed on multiple PS to exploit model-parallelism, since the size of embedding parameters simply prevents model replication. To maximize training throughput, the parameters of embedding operators are updated using Hogwild! [51]. In addition, the readers are deployed on a separate tier of machines to feed training batches to the trainers as illustrated in Fig. 3.

Such PS-based system is well suited for DLRMs allowing scaling different components separately and achieving a balanced resource utilization when training different models with different trainer, parameter server and reader configurations. Moreover, resources in the system are largely fungible, making it low-cost for datacenter operations.

However, the need for supporting DLRMs with trillions of parameters and therefore terabytes in size poses a serious challenge to

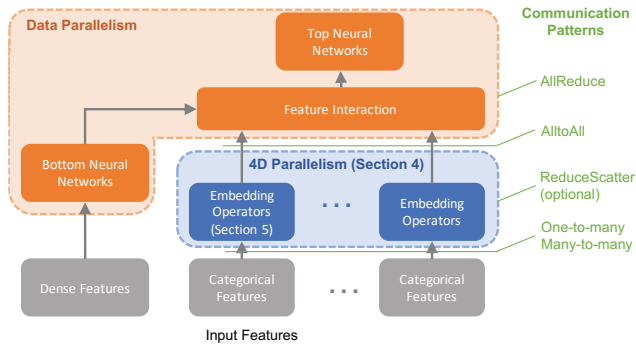


Figure 4: Neo overview. Each box in the figure indicates a neural network component, while edges between boxes are tensors shared between different components.

the scalability of this approach, necessitating a steep increase of the number of trainers and parameter-servers to meet the ever growing training requirements. This quickly becomes intractable, degrading model accuracy with staleness due to increased asynchronous updates across a very large number of workers. To tackle these issues, we build a high-performance synchronous training solution for large DLRMs, decoupling distributed scaling from statistical quality.

The efficient design of the synchronous training system leads us to use a novel combination of *4D parallelism* (Section 4) for memory intensive embeddings tables, data parallelism for compute intensive DNN operators, and pipelining across different components. This hybrid parallelism requires AlltoAll communications for the embedding lookup results [42, 43], as well as embedding table input redistribution if the inputs are streamed from database in batches, which is often the case. Unlike AllReduce communications for gradient synchronizations, which can be overlapped, these AlltoAll communications are on the critical path due to data dependencies, stressing the performance of the interconnect and communication primitives. Furthermore DLRMs are typically trained on *very large* amounts of data, which corresponds to mostly unstructured and unlabeled interactions from a wide variety of applications. Typical data-set sizes are in the range of several petabytes, necessitating the use of common, distributed network storage, such as the Tectonic filesystem [46]. For training, this data would need to be streamed in, putting additional stress on the host network and host-to-device bandwidth.

3 OVERVIEW

Fig. 4 shows an overview of Neo, a software-hardware co-designed system for fast and scalable training of DLRMs. This section briefly describes the key components of Neo.

First, Neo uses *data parallelism* for training compute-intensive DNN layers (shown in orange) and switches to a *4D parallelism* strategy that combines *table-wise*, *row-wise*, *column-wise*, and *data* parallelism for efficient training of memory-intensive embedding operators.

Second, Neo is equipped with a high-performance implementation for embedding operators. This is achieved by a number of

critical systems optimizations, including (1) a *hybrid kernel fusion* technique to reduce the computational cost of embedding operators, (2) a *software-managed caching* mechanism to leverage heterogeneous memories of modern hardware platforms, and (3) a variety of *quality-preserving compression* techniques to minimize the memory requirement for embedding computation.

Finally, Neo is deployed on *ZionEX*, a new hardware platform co-designed with Neo’s 4D parallelism to optimize inter-node communications for DLRM training.

Additionally, data I/O is an integral part of any training system, especially with the adoption of fully synchronous training and accelerators. First, the host to device transfer should be non-blocking and fast enough not to limit the overall training throughput. Ideally overlapping the input data transfers with training using double buffering or pipelining. Second, even though mapping input data distribution to collective communications between trainers is faster, this introduces additional challenges for the input and output data layout of the collective communications. Initial experiments show that these could add significant latency to the critical path. We will illustrate how we overcome these practical challenges in Section 7.1.

4 4D PARALLELISM

A key component in DLRM is embedding operators, which will be defined in Section 5. To enable high-performance training for embedding operators, it is crucial to effectively balance the workload distribution across GPUs and minimize communication costs. We introduce *4D parallelism*, which combines *table-wise*, *row-wise*, *column-wise*, and *data* parallelism for jointly optimizing the parallelization performance of embedding operators.

Table-wise parallelism. The most straightforward parallelism scheme is partitioning and parallelizing multiple embedding tables across GPUs, as shown in Figure 5a. Table-wise parallelism does not further split embedding tables, therefore this scheme requires no additional handling of embedding table input indices or pooled embedding results, leading to optimal communication efficiency. However, table-wise parallelism cannot handle large embedding tables that exceed the memory capacity of a single GPU, and the achieved load balance is often limited due to the skew in table sizes.

Row-wise parallelism. This scheme parallelizes large embedding tables by rows and assigning different table shards to different trainers. Since the embedding table inputs index tables by rows, they need to be *bucketized* based on the row-wise parallelism decision and distributed to the respective trainers, as illustrated in Figure 5b. Moreover, partial results on multiple trainers need to be reduced and then scattered to all trainers for downstream computations. This requires a *ReduceScatter* communication pattern in the forward pass. This scheme handles large tables well and leads to better load balance. However, the communication cost scales linearly with the number of trainers.

Column-wise parallelism. Column-wise parallelism partitions the embedding tables along the embedding dimensions (see Figure 5c) and treats the partitioned table with smaller embedding dimensions as individual operators. This scheme requires duplication

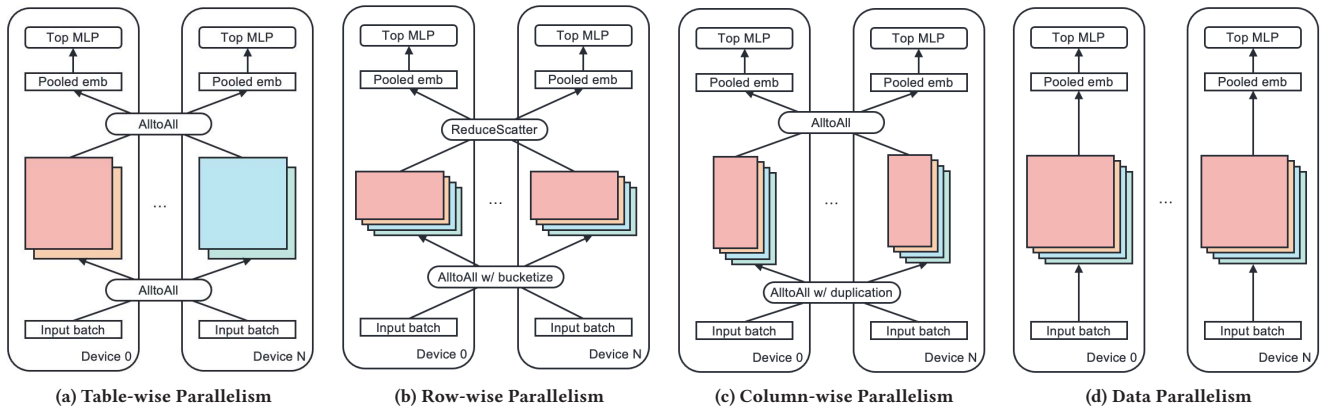


Figure 5: Embedding table sharding schemes with different implications on the communication cost, load balancing and memory requirement. Bottom MLP is omitted in this figure for simplicity of illustration.

of input indices for the partitioned tables. Compared with table-wise parallelism, it preserves the same flow and communication pattern (AlltoAll). A key advantage of column-wise parallelism is enabling finer-grained parallelism, especially for large tables. However, it works well only with large embedding dimensions and increases the payload for the input indices, which have to be replicated to all nodes with the column shards. Furthermore, since the rows of column-wise sharded tables are split across different trainers, using an independent row-wise update for these tables introduces additional parameters, one for each shard of the row instead of just a single value for the entire row when using sparse optimizers (see Section 5.1 for details).

Data parallelism. DLRMs tend to have a wide range of table sizes, while table-, row-, and column-wise parallelism are efficient for relatively large embedding tables prohibitive to replicate. For smaller tables, data parallelism achieves better performance, since data parallelism does not involve any communication in the forward pass (see Figure 5d). Therefore, for small embedding tables, Neo treats embedding tables as dense parameters and replicate them across all trainers. AlltoAll is no longer needed for the pooled embeddings of data-parallel embedding tables. Instead, AllReduce is required to synchronize across all replicas. As a result, this depends on the trade-off between the cost of AlltoAll of the pooled embeddings versus the cost of AllReduce on the entire table. In general, small embedding tables with fewer rows are good candidates for data parallelism. Input indices for these tables are passed through as data-parallel inputs and no longer require re-distribution.

4.1 Parallelization Algorithms

Neo supports applying 4D parallelism strategies at the granularity of individual embedding operators to maximize flexibility. Practitioners can mix-and-match the above primitives to determine the best strategy to partition an embedding operator. Additionally, Neo also supports partitioning embedding operators in a *recursive* manner at different levels of hardware hierarchy to further improve workload balance and hardware efficiency. For example, the

table-wise then row-wise scheme first assigns a set of tables to a particular node, and within that node the tables are partitioned row-wise. This family of hierarchical parallelism schemes improve hardware locality by fully exploiting the fast GPU interconnects and reduce inter-node communications.

With a cost function defined for each of the above parallelism schemes, placement algorithms can be explored to minimize the cost differences between workers. The cost function is a combination of communication overhead and load imbalance between the trainers. The communication overheads are computed using the message volume as a representative metric, with higher message volumes corresponding to higher costs. This is largely accurate in capturing the throughput costs and for latency measured values are incorporated as a fixed additive cost. We estimate the load imbalance by using the embedding access size per trainer, which can be approximated as the number of embedding tables per trainer $\times$ the global batch size $\times$ average number of indices per sample $\times$ embedding dimension. The combination of both costs gives us a reasonable estimate for communication and load imbalance. Further we introduce scalar weight for each of the individual costs, which can be tuned based on different system specs to get more accurate estimations.

We implement and evaluate two polynomial time heuristics as a proof of concept. The first one is a simple greedy heuristic that sorts the costs of available schemes in a descending order and allocates the largest shard first, one per worker. Then, the greedy algorithm iterates through all remaining shards and assigns the top cost to the node with the smallest sum of costs. A second heuristic is the largest differencing method (also known as the Karmarkar–Karp algorithm [26]). The main idea is to take the two largest numbers from the input and replace them by their difference. It directly reduces the difference of sums and generally outperforms the greedy heuristic.

4.2 Pipelining

Although using GPUs as the main compute resource offers limited pipelining opportunities within model evaluation, we improve GPU

utilization by pipelining inter-batch data movement and overlapping communication with computation.

When batch i is being evaluated, the same GPUs can start receiving and distributing batch $i + 1$ using a separate stream. To minimize the interference, we overlap the input All-to-All of batch $i + 1$ with the forward propagation of top MLP of batch i where no communication is involved. In addition, we overlap the pooled embedding All-to-All with the forward propagation of bottom MLP to hide latency.

5 EMBEDDING OPTIMIZATIONS

Optimizing the runtime performance of DLRM’s embedding operators (see Section 2) requires addressing two key challenges. First, the forward processing, backward propagation, and gradient updates for the embedding operators require launching thousands of GPU kernels in each training iteration, introducing significant GPU kernel launch overhead. Second, some embedding operators may include up to billions of parameters and do not fit on the device memory of a single GPU.

We introduce three novel techniques to reduce the computational cost and memory requirement of embedding operators. First, we introduce a *hybrid kernel fusion* technique to minimize the CUDA kernel launch overhead and allow each GPU worker to only launch two kernels (i.e., one for forward and one for back propagation and parameter update). Second, for parallelizing the computation of the embedding operators, we propose *column-wise parallelism* and *row-wise parallelism* in addition to data and model parallelism. The combinations of these four parallelism dimensions enable Neo to support embedding tables with up to trillions of parameters. Finally, Neo exploits a series of memory saving techniques that leverage the memory hierarchy of the *ZionEX* platform to ensure sufficient memory capacity for DLRM.

5.1 Kernel Fusion

Neo uses a *hybrid kernel fusion* mechanism to minimize the CUDA kernel launch overhead for performing embedding computations in a training iteration. First, instead of applying a separate embedding lookup for each embedding table, Neo fuses multiple embedding lookups on the same GPU into a single CUDA kernel (Figure 6a), which improves the parallelism and bandwidth utilization and reduces the overhead of launching multiple CUDA kernels on GPUs.

Second, Neo also fuses the backward pass with the sparse optimizer to further reduce kernel launch overhead and avoid materializing gradients to the embedding tables. The key challenge of such fusion is avoiding potential race-condition across gradient updates from different training samples and handling non-linearity in advanced optimizers such as AdaGrad [11], LAMB [66], and Adam [27]. For example, both sample 1 and 2 in Figure 2 contribute to the gradients of the embedding vector 1 and 6. Directly sending these gradients to a non-linear sparse optimizer without aggregation would result in incorrect updates to the embedding tables.

To guarantee correctness while maximizing performance, Neo applies *gradient sorting* by rows so that gradients to the same embedding rows are processed by a single CUDA thread block, as shown in Figure 6b. *Gradient aggregation* is subsequently applied

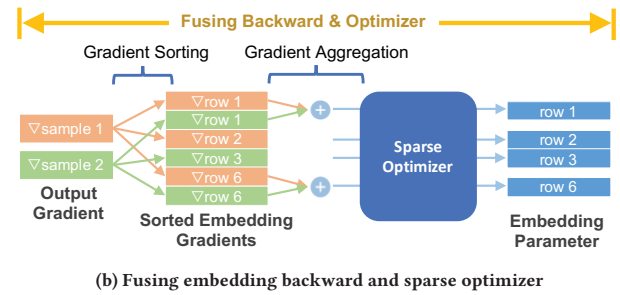
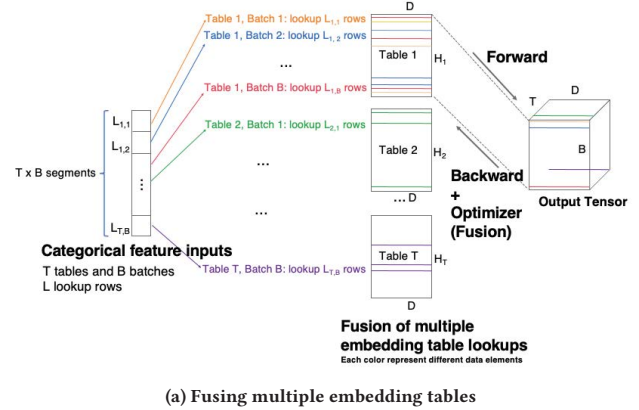


Figure 6: Embedding operator optimizations

within each CUDA thread block using much faster but smaller GPU shared memory.

Neo’s hybrid fusion technique for embedding operators lead to three performance benefits. First, Neo reduces the memory requirement for embedding operators by avoiding allocating GPU device memory for embedding gradients. Second, the memory accesses to GPU device memory are minimized by using GPU shared memory to save intermediate embedding gradients. Finally, kernel fusion improves the overall performance of embedding computations by up to 7× compared to a native implementation. The optimized embedding operator implementations are open sourced as part of the FBGEMM library* and integrated with PyTorch.

5.2 Managing Memory Hierarchy

For DLRMs with up to trillions of parameters, the embedding tables are too large to entirely fit on a single GPU. We leverage multiple levels of memory hierarchy of the *ZionEX* platform, including HBM, DRAM and SSDs in addition to scaling out to multiple nodes for increasing aggregate capacity, to ensure sufficient memory for the models, with the faster memory serving as a software cache of the subsequent layer. Neo’s hierarchical memory management strategy is specifically useful for online training of DLRMs, which warrants using fewer nodes for training original large models, due to lower throughput requirements, as outlined in Sec. 2.

*PyTorch FBGEMM_GPU library: https://github.com/pytorch/fbgemm/tree/master/fbgemm_gpu

One approach to managing memory hierarchy is CUDA’s unified memory (UVM) [44], which provides a single memory address space for different memory types and automatically replaces and evicts unused pages. However, random table lookups in embedding operators requires caching and replacing unused parameters at the granularity of individual embedding rows, which makes using UVM as-is insufficient for DLRM. Necessitating additional handling of the look-up to ensure performance is not bound by the frequent host to device transfers. Instead, Neo uses a customized 32-way set-associative software cache [64] using least recently used (LRU) or least frequently used (LFU) cache replacement policies, where the associativity matches the warp size of GPUs. This enables fine grain control of caching and replacement, allowing it to be tuned for target model characteristics. Note that UVM is bounded by PCIe bandwidth, while Neo’s software cache can bridge the gap for the bandwidth between PCIe and HBM (50× difference). The software cache improves the end-to-end performance of DLRM workloads by approximately 15% compared to UVM.

To further reduce the memory requirement of embedding operators, Neo also employs a variety of compression techniques introduced in prior work, such as a row-wise sparse optimizer [14, 62], low/mixed-precision training using a high-precision cache backed by low precision embedding tables [63], and advanced factorization techniques [29].

The row-wise sparse AdaGrad was first introduced in [14], and then further elaborated in [62]. In the row-wise sparse AdaGrad, each element of the moment estimation is applied to the entire embedding row. For each row it is a single scaling factor that is updated by adding the average squared sum of gradients across the row. In this way, we keep the momentum as a 1D tensor with H elements instead of $H \times D$ 2D tensor, where H and D are the number of rows and the number of elements per row in an embedding table, respectively.

6 ZIONEX: HARDWARE PLATFORM DESIGN

We begin by describing the limitations of our previous hardware platform for DLRM in Section 6.1. Section 6.2 introduces *ZionEX*, a new hardware platform for DLRM. We also outline the design principles used in the development of *ZionEX*.

6.1 Previous Platform: *Zion*

Zion [40] introduced in 2019 was our previous work aimed as a high-performance hardware platform for training DLRMs. While *Zion* offers significantly improved capabilities at single-node level, it falls short as a distributed platform not being extensible to meet the rapidly growing DLRM training requirements. We critically appraise its limitations, but other platforms based on a similar design share the same limitations; we discuss those platforms in Section 9.

Figure 7a shows the architecture of a *Zion* node, which has 8 CPU sockets with 1.5 TB memory, 8 GPUs, and 8 network interface cards (NICs). It provides a powerful heterogeneous *super node* design for training DLRM by (1) offloading compute heavy layers of DLRM (e.g., MLPs) onto GPUs and (2) leveraging CPUs for large embedding operators on the relatively cheaper DRAM instead of HBM for accommodating TB-scale DLRMs on a single node.

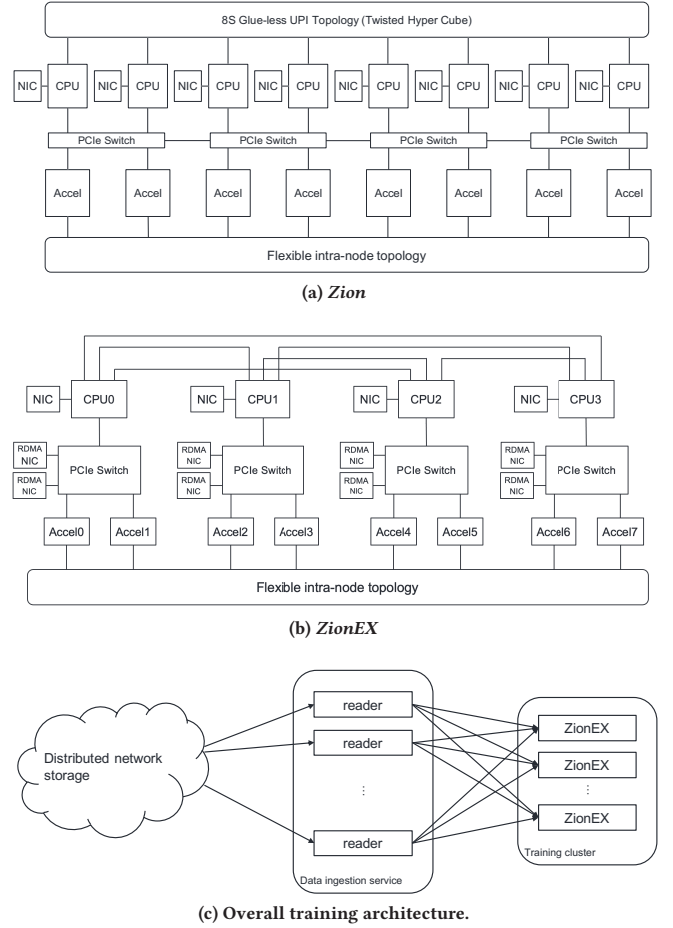


Figure 7: The system architectures of *Zion*, *ZionEX* platforms and the overall training system.

However, this heterogeneous design introduces a number of challenges to software design and performance. For example, it’s critical to balance the workload on CPUs and GPUs to ensure maximum overlap. This requires elaborate pipelining between CPUs and GPUs and partitioning DLRM into fine-grained tasks using an accurate cost model. In addition, heterogeneous training of DLRM also introduces non-trivial runtime overheads, such as increased data transfers between CPUs and GPUs and inter-socket communication.

Finally, a critical missing component in *Zion* is that each NIC is directly attached to a CPU. As a result, all of the inter-node communications (e.g., gradient synchronization and tensor transformation) necessitate CPU intervention and additional GPU-CPU transfers. Furthermore these NICs are connected to the common shared data-center network infrastructure, which introduces overheads and interference from network congestion, and are constrained to use more data center-friendly topologies, protocols (TCP/IP) which are sub-optimal for distributed training. Although each *Zion* node is equipped with 8x 100Gbps NIC bandwidth, in reality we found it is very difficult to scale out to multiple nodes due to networking

overheads. With today’s increasing demand on modeling size of DLRMs, *Zion* is not able to scale well and fully utilize the powerful hardware resources.

6.2 *ZionEX*

To address these shortcomings, we introduce *ZionEX*, which we have designed to be more scalable than the previous *Zion* platform with improved network capabilities, while retaining its flexibility and core advantages, such as the OAM form factor, modular design [40, 57], and flexible intra-node accelerator fabric [69]. With all of these improvements *ZionEX* bring about orders of magnitude higher capability both in terms of supporting increased model complexity and higher training performance. This is best illustrated by comparing the product of maximal model complexity (in terms of FLOPS/sample) supported by each platform and achieved training throughput, which can be seen as normalized effective performance. For *ZionEX* with achieving a throughput of 1.2 MQPS for a model with 638 MFLOPS/sample (from Table.3), this translates into a effective performance of 766 TFLOPS/s, with additional headroom to go up to several PETAFLIPS/s. Whereas for *Zion*, the maximal model complexity that could be supported was less half of that on *ZionEX* ($\approx 250\text{MFLOPS/sample}$) and with much lower throughput ($\approx 0.25\text{MQPS}$)[4, 42], thereby leading to more than $10\times$ lower max achievable effective performance of only 63 TFLOPS/s. Figure 7b shows the overall system architecture. We briefly highlight *ZionEX*’s core design principles:

Scalability. Both *Zion* and *ZionEX* support heterogeneous training of DLRM, but the most striking difference is that *ZionEX* is designed with sufficient scale-up and scale-out network capabilities. As shown in Figure 7b, *ZionEX* employs a dedicated *RDMA over Converged Ethernet* (RoCE) NIC for each of the GPUs connected via PCIe switches to allow for a dedicated inter-node connectivity (isolated from common data-center network) and importantly support more efficient RDMA/GPUDirect communication protocols [42]. These *ZionEX* nodes can be connected with a dedicated backend network to form a cluster for distributed scalable training. The extensible design of *ZionEX* allows for scaling the backend network to interconnect many thousands of nodes, forming a *data-center scale AI training cluster*.

High Performance. As a scale-out solution, we offload the entire DLRM to GPUs to fully leverage the massive parallelism and high memory bandwidth to accelerate MLPs and embedding computations. To transfer tensors and synchronize gradients, each GPU can communicate directly with GPUs on a different node through the dedicated low-latency high-bandwidth RoCE NIC, without involving host CPUs. In addition, *ZionEX* also has a frontend NIC connected to each CPU. Data ingestion goes through the regular frontend network and PCIe, without interfering with activations or gradients. The host CPUs are only used to setup input batches and marshal the training process.

Capability. With *ZionEX* we ensure that the platform is compatible with existing infrastructure and can be widely deployed within our data-centers, without causing major disruptions. This is critical for being able to effectively leverage the capability of the platform and make it readily available to across variety of applications and

uses-cases. We achieve this by making the *ZionEX* platform compliant with the standard Open Rack specifications [2], which covers the compatibility with other infrastructure components such as power, cooling, mechanicals and cabling. Furthermore designing the platform to be modular and relying on open standards based technologies, for instance - the ethernet based network fabric for high-performance scale out solution.

Fig. 7c shows the overall training platform, along with the disaggregated data-ingestion service. This supports streaming input data from a network store such as Tectonic [46] and perform lightweight data pre-processing operations in a distributed fashion. So that the data-ingestion is not a bottleneck for the end-to-end training and to ensure sufficient throughput in feeding *ZionEX* trainers.

7 IMPLEMENTATION

We detail the implementation of high-performance scalable training for DLRMs described above. We built a high-performance training software stack for DLRMs using PyTorch [48], with efficient CUDA implementation for most deep learning operators via the ATen library, and automatic handling of parameter replication and gradient synchronization with overlapped back-propagation and AllReduce via the PyTorch DistributedDataParallel library [32]. We have enabled the following components for efficient DLRM training.

7.1 Data ingestion

Data ingestion is a key component to ensure end-to-end training performance especially for DLRMs, which typically process through order(s) of magnitude larger amount of data than other typical DNN models. We observe that data ingestion, if left unoptimized, can incur significant latency and introduce non-trivial overheads to pipelining.

Originally designed for a distributed asynchronous CPU setup, our readers and data pre-processing module stores the offsets and indices[†] of each sparse feature in separate tensors per embedding table. As a result, a DLRM with hundreds of embedding tables can easily get a thousand input tensors per iteration, which translates into significant overheads from *CPU* $\leftrightarrow$ *GPU* transfers and was one of the key bottlenecks for the previous *Zion* platform as detailed in Sec. 2.

To overcome this practical challenge, we co-designed the data pre-processing module to use a combined format where lengths rather than offsets are used and inputs to different embedding tables are simply concatenated. The benefits of using the combined format are two-fold: (1) it optimizes CPU-GPU transfer by consolidating small transfers; (2) it can be directly consumed by the embedding kernel without additional layout transformations. We further optimized input data transfer by using pinned memory to avoid the extra copy.

With the combined format, we developed a module to efficiently distribute embedding table inputs based on the sharding strategy. In the case of table-wise sharding (shown in Fig. 5a), an AlltoAll is needed to distribute the global batch for local tables to each worker. Since the size of indices is dependent on the values of the lengths, the communication is actually implemented as an AlltoAll for

[†]Please refer to the interface of `nn.EmbeddingBag` <https://pytorch.org/docs/stable/generated/torch.nn.EmbeddingBag.html>

Table 2: ZionEX per-node system specification.

Compute (PFLOPS)	2.5 (FP32) / 20 (TF32) / 40 (FP16,BF16)
HBM	320 GB, 12.4 TB/s
DDR	1.5 TB, 320 GB/s
Scale-up bandwidth	2.4 TB/s (uni-directional)
Scale-out bandwidth	1600 Gbps (uni-directional)
Host NW	4 × 100 Gbps

lengths followed by an AlltoAll for indices. In a setup with W workers, T local tables and B local batch size, this gives us indices in the order of (W, T, B) , which needs to be further permuted to (T, W, B) for embedding kernel consumption. We have developed custom GPU kernels for permute, bucketize and replicate to achieve maximum throughput on embedding input indices distribution for table-wise, row-wise and column-wise sharding schemes. Check-pointing the model has similar challenges, requiring to be sufficiently frequency be able to write-out such larger model whilst not becoming an overhead for training, as outlined in this recent paper [12].

7.2 Communication Primitives

High-performance collective communication is key to performant and scalable DLRM training. PyTorch provides the Process Group (PG) interface for collectives - an abstract platform / collectives library agnostic API. DLRM uses this API directly (for Alltoall) or indirectly via DDP (for Allreduce) [32]. We use the NVIDIA’s *Collective Communication Library* (NCCL) as our primary collective communication library since it efficiently uses RDMA and NVLINK for best performance. We extended PyTorch NCCL process group implementation to support Alltoall/Alltoallv collectives using NCCL Send/Recv primitives (requires NCCL 2.7.3 or later).

8 EVALUATION

We provide results for end-to-end training of production models, operator-wise performance breakdown.

8.1 Experimental Setup

Table 2 summarizes the aggregated capabilities of a single ZionEX node with 8 NVIDIA A100 GPUs. The 8 GPUs in a node provide a total 320 GB HBM with 12.4 TB/s aggregated memory bandwidth. The 4-socket CPUs provide 1.5 TB memory with 320 GB/s bandwidth. On network capabilities, the GPUs are interconnected with high-bandwidth NVLink for intra-node GPU communication, and each GPU has a dedicated 200 Gbps RoCE NIC for inter-node communication. We use a cluster of 16 ZionEX nodes in the experiments with 5TB total HBM capacity.

8.2 End-to-End Training

We report results on three DLRMs deployed in production for different tasks, including click through rate (CTR) prediction, ranking, and engagement. Table 3 lists high-level characteristics of these candidate models. Model-A represents large and complex DLRMs that

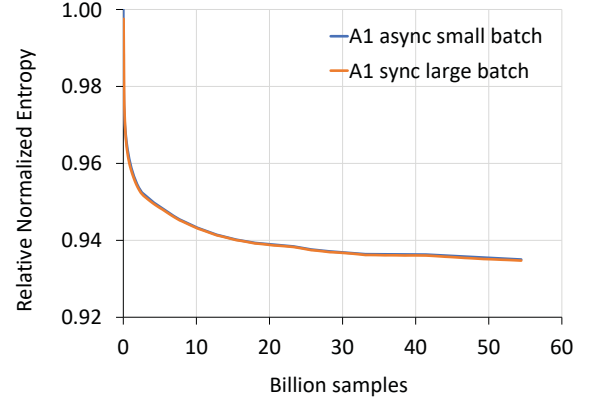


Figure 8: Training quality comparison between asynchronous small batch on a distributed CPU platform and synchronous large batch on the proposed platform, measured in relative normalized entropy [20].

stress Neo’s compute capability and communication bandwidth, using significantly higher FLOPS per sample and a large number of embeddings. Model-F presents a different practical challenge where despite having low FLOPS per sample and a small number of embedding tables, it has a single massive table that cannot fit in the device memory of a single GPU. Finally, Model-I represents moderate scale DLRMs stressing memory bandwidth with high average embedding pooling sizes. These target models are trained on up to 16 ZionEX nodes (128 GPUs) in the cluster. The model qualities are evaluated in normalized entropy [20], and the training throughput is measured in queries per second (QPS).

First, we use model-A to demonstrate the training quality, since it can also be trained on a distributed CPU platform. As shown in Figure 8, despite using significantly larger batch size (64K vs. ~150), synchronous large batch training on ZionEX provides on-par or better model quality (both using tuned hyperparameters). With the same configuration, Neo achieves 1.2 MQPS using 128 GPUs on 16 nodes, a 40× speedup compared to our previous generation

Table 3: Target models configuration

Model	model-A	model-F	model-I
Num parameters	793B	12T	332B
MFLOPS per sample	638	5	60
Num of emb tables	≈ 1000s	≈ 10s	≈ 100s
Embedding table dims (range [min, max], avg)	[4, 384] avg: 93	[256, 256] avg: 256	[92, 92] avg: 92
Avg pooling size	15	20	70
Num MLP layers	20	7	43
Avg MLP size	3375	490	682
Target local batch size	512	512	2048
Achieved QPS	1.2M	1.7M	3.4M

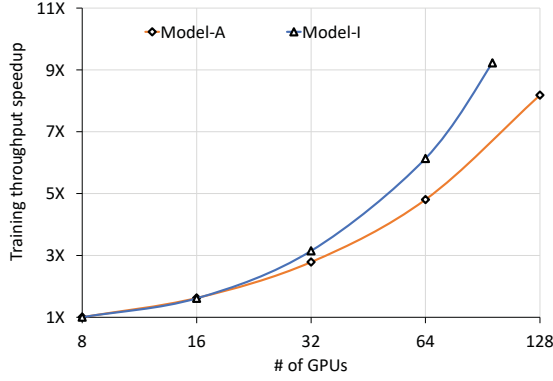


Figure 9: Training throughput scaling for model-A and model-I, relative to 8 GPUs (1 node).

distributed CPU asynchronous training platform using 45 parameter servers and 15 trainers. While previous solution was unable to scale out further without hurting training quality, fully synchronous training on *ZionEX* allows scaling beyond 16 nodes with even larger batch sizes.

8.3 Scaling Performance

Figure 9 shows the normalized training throughput of model-A and model-I using up to 16 nodes, while keeping the per-GPU batch size constant. While the workload of data-parallel training remains the same with the scaling, the numbers of embedding tables per GPU reduces with scaling due to model-parallelism. For the same reason, however, each GPU processes the entire global minibatch for each of its local tables and this increases commensurately with scale and compensating for the reduced tables, making this still a weak scaling experiment. To run on smaller node counts, we reduce the embedding table cardinality and hash inputs to be within the reduced number of rows. This *shrunk version of the model* effectively reduces the model sizes with minimal/no impact on the performance characteristics, hence is used for studying scaling performance.

As seen from the figure, on larger node counts, the scaling efficiency is around 50% for model-A and around 75% for model-I. While model-A and model-I come very close in terms of effective FLOPS and memory requirements after considering the target local batch size, model-A has larger fully exposed AlltoAll latency. This is because more embedding tables increase AlltoAll payload, and mixed dimensions make it more difficult to balance embedding computations and AlltoAll communications at the same time. As a consequence, model-A suffers more from reduced AlltoAll efficiency when scaling out.

To better understand the scaling performance, we provide a breakdown of serialized and exposed training iteration latency of model-A in Figure 10. Comparing between serialized and exposed latency, the CPU to GPU transfer (i.e., HtoD) is completely hidden, and the exposed communication latency is much less than serialized AlltoAll and AllReduce latency combined. This demonstrates the effectiveness of Neo’s pipelining optimization to overlap communications with computations (see Section 4.2).

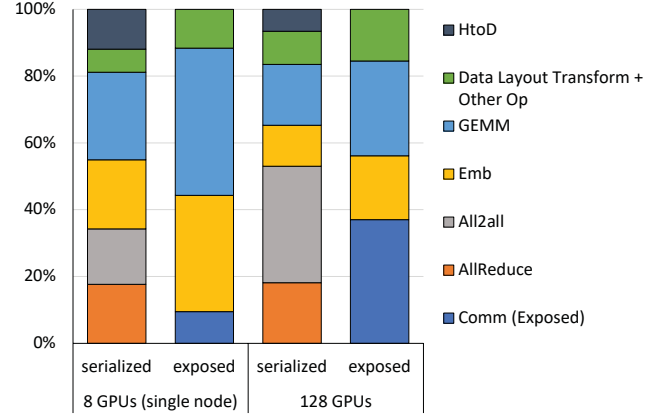


Figure 10: Model-A (with local batch size per GPU = 512) results and dominant operator time breakdown (serialized and exposed time) per GPU, after optimizations.

As node count increases, we observe increased AlltoAll and AllReduce latencies. Since most AlltoAll communications are on the critical path, increased AlltoAll cost has a direct impact on the exposed communication and overall training latency. While AllReduce is mostly hidden on up to 16 nodes, the increased AllReduce latency and unchanged computation latency signifies that AllReduce can become the bottleneck once the slack in backward pass is completely used up with higher node counts and/or faster computation.

8.4 Training Throughput Optimizations

Using model-A as a case study, we detail the various optimizations and their contributions in achieving up to 1.5 MQPS, shown in Figure 11. Further, we use the performance roofline modeling methodology described in Appendix-B to establish the upper bound of achievable performance and confirm that reported throughput is within 15% theoretical estimates. The baseline performance for model-A on 128 GPUs is below 700 KQPS. Further profiling reveals large disparities on embedding lookup latency between different GPUs, signifying severe load imbalance. This is mitigated using a combination of table-wise, column-wise, and data parallelism for the ≈ 1000 s of embedding tables to partition them across 128 GPUs. Note that even though column-wise parallelism introduces additional cost to its input AlltoAll, the benefit from better load-balance outweighs the overheads and results in overall QPS improvement by 20%. However, the scaling efficiency is still about 30% lower than ideal linear scaling.

As discussed previously, the two major issues limiting scaling efficiency are: (1) load imbalance and (2) increased AlltoAll latency. For model-A, further balancing the load using only HBM is particularly challenging because the model size in TF32 comes close to the 5TB aggregated HBM capacity on 128 GPUs. After discounting for memory reserved by PyTorch framework and NCCL on each rank, Neo has little room to explore placement strategies. To mitigate this issue, we use lower precision (FP16) embedding tables [67], reducing the model size by a factor of 2. While this alone does not provide direct throughput benefit, Neo can leverage the head room to strike a better balance. As a consequence, the

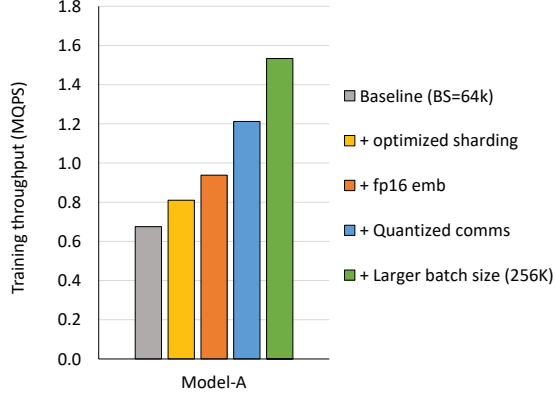


Figure 11: Training throughput improvements enabled by optimized sharding, reduced precision embeddings, quantized communications and larger batch sizes.

training throughput is increased by another 20% due to improved load balancing.

Next, to address the increased AlltoAll latency, we incorporate quantized collective communications proposed in [65], which directly reduce the communication volume. For model-A, we validate that using FP16 in forward AlltoAll and BF16 in backward AlltoAll provides almost 30% speedup without any training quality loss.

Lastly, we increase the global batch size from 64K to 256K. This directly increases activation sizes, which helps saturate GPUs and communication bandwidth better, while being complimentary to all other optimizations. With appropriately tuned optimizer/hyperparameters, we are able to achieve on-par training quality, however more comprehensive experimentation is warranted since large batch training of DLRLMs is not as well studied and will be part of future work. Collectively, these techniques unlock an 87% improvement on training throughput compared to TF32 training with a 64K global batch size.

8.5 Model Capacity Limitation Study

We use model-F as an example to push the model capacity on the prototype system. Unlike model-A or model-I, efficiently training model-F presents 2 different challenges. First, with 12T parameters, model-F can easily require up to 96TB of memory using a naive training approach, far exceeding the total memory available on a 16-node cluster[‡]. Second, the model has only a few massive embedding tables with ~10B rows and 256 columns, each requiring multi-node worth of GPU and host memory to train.

To fit the model onto 16 nodes, we first apply row-wise sparse AdaGrad optimizer to embedding tables which reduces optimizer states from per element to per embedding row. Then we use FP16 precision on embedding tables [67]. These two optimizations collectively bring model memory footprint from 96TB down to 24TB, just fitting under the 4TB HBM + 24TB DRAM memory hierarchy. On the massive embedding tables, we enable row-wise sharding

to distribute the tables to multiple nodes and adjust the training flow to use AlltoAll with bucketization and ReduceScatter as shown in Figure 5b. With UVM enabled and HBM used as a cache, we are able to train model-F with throughput as high as 1.7 MQPS, demonstrating capability of our HW/SW co-designed solution to push beyond the current state-of-the-art.

9 RELATED WORK

Researchers have proposed various system-level innovations to tackle the challenges from extremely large models. DeepSpeed [50] fully shards model parameters, gradients and optimizer states across all nodes, and reconstructs necessary states on the fly using checkpoint partitioning and rematerialization [21, 28] to drastically reduce memory usage. GShard [31] trains a massive translation model with mixture of experts, sharded across accelerators through annotation of parallelization strategy at tensor level. FlexFlow [22] uses automatic search to discover the best operator parallelization strategy in the graph. Building on this direction of auto-parallelization, these recent papers [39, 60] use optimal synthesis and reinforcement learning to find optimized device placement to further improve parallelism without the need for manual intervention. However, these general systems are not specifically designed for highly sparse recommendation models.

To that end, Alibaba introduced XDL [23], an industry-scale training system designed for high-dimensional sparse data. XDL incorporates optimizations such as hierarchical sample compression, workflow pipelining, zero copy and CPU binding to improve training efficiency of the sparse part of the model. Kraken [62] targets at more efficient online training with decoupled key-value fetching and embedding, codesigned cache eviction policy with ML domain knowledge for the embedding tables, memory efficient optimizers for the sparse and dense part of the model, and a non-located deployment model allowing the inference servers and parameter servers to grow independently. [25] optimizes CPU-based DLRLM training through lock-free embedding table update, tuned loop tiling for dense MLP, the AlltoAll communication primitive and a new split-SGD implementation that takes advantage of the bits aliasing in FP32 and BFloat16 to reduce memory footprint. Baidu’s AIBox [70] takes a different approach to horizontally scaling and focuses on fitting training of large recommendation models in a single node. AIBox hides serving latency by pipelining network, disk and CPU/GPU tasks, reduces model update overhead, and improves SSD life span through a grouped hashing scheme and a multi-level in-memory hashing system.

Much attention is given to communication performance as it has become a major bottleneck in distributed training at cluster and datacenter scale. BytePS and ByteScheduler [24, 49] harnesses idle CPU and network resources and better communication scheduling to improve parameter exchange efficiency. However, in a homogeneous training cluster where each job spans multiple nodes, there are reduced opportunities for finding and exploiting spare network resources, resulting in a sub-optimal use of such approach. SwitchML and ATP [30, 53] leverages programmable network switches to perform in-network aggregation for cross-rack bandwidth reduction in datacenter environments. [6, 36] discovers and exploits datacenter network locality and forms optimized and dynamic aggregation

[‡]Considering FP32 precision and doubled size for optimizer states $12e12 \times 4 \times 2 = 96e12$. The prototype cluster has in total 4TB HBM and 24TB DRAM

routes through learning and optimal synthesis. Alternatively, these papers [33, 34] address the communication overheads by using various quantization schemes to reduce communication volume.

10 CONCLUSION

DLRMs are an important class of models widely used by many internet companies for a wide range of applications. They can often be the single largest AI application in terms of infrastructure demand in data-centers. These models have atypical requirements compared to other types of deep learning models, but they still follow a similar trend of rapid rate of growth that is common across all deep learning-based applications. This growth constantly pushes the performance boundary required of the underlying software stack and hardware platform.

In this paper we co-design a solution that enables us to run models with trillions of parameters, while attaining 40× faster total training time for production recommendation models. On the software side, Neo is equipped with a number of novel software techniques, including 4D parallelism, high-performance embedding kernels, hybrid kernel fusion, and hierarchical memory management. On the hardware side, the extensible *ZionEX* platform allows for scaling up to the full data center with thousands of nodes, thus enabling a data center-scale AI training cluster to continue catering to the growing demands of deep learning models.

Finally, we also explore co-designing models and algorithms to make them more amenable to the training cluster, for instance model architectures that reduce global AlltoAll communication for better scaling efficiency. With this solution successfully deployed in production, we intend to continue working on these future directions to further push the capability for large scale deep learning training.

ACKNOWLEDGEMENTS

We would like to acknowledge all of the help from members of the hardware, datacenter and infrastructure teams, without which we could not have achieved any of the above reported results. This includes among others *Jenny Yu, Matt Hoover, Hao Shen, Damien Chong, Jeff Puglis, Garnnet Thompson, Peter Bracewell, Anthony Chan, Wei Zhang, Michael Haken, Tiffany Jin, Joshua Held, Cheng Chen, Yin Hang, Ben Kim, Tyler Hart, Gada Badeer, Ahmed Qaid, Peichen Chang, Zhengyu Yang, Anil Agrawal, Viswesh Sankaran, Daniel Montgomery, James Taylor, Jeff Anderson, Amithash Prasad, Patrick Williams, Harsha Bojja, Arrow Luo, Changduk Kim, James Le, Rachel W Wang, Vignesh Mathimohan, Shockely Chen, Doug Wimer, James Allen, Vidya Rajasekaran, Kelly Zuckerman, Wenyin Fu, Valentin Andrei, Matt Skach, Philipp Keller, Olivier Raginel, Danielle Costantino*. We also like to thank other reviewers who have gone through multiple drafts of this paper, providing helpful inputs.

REFERENCES

- [1] [n.d.]. NVIDIA Collective Communications Library (NCCL), <https://developer.nvidia.com/nccl>.
- [2] [n.d.]. OCP Open rack standard (v2), https://www.opencompute.org/wiki/Open_Rack/SpecsAndDesigns#RACK_Standards.
- [3] Martin Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2015. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. <https://www.tensorflow.org/>. Software available from tensorflow.org.
- [4] Bilge Acun, Matthew Murphy, Xiaodong Wang, Jade Nie, Carole-Jean Wu, and Kim Hazelwood. 2020. Understanding Training Efficiency of Deep Learning Recommendation Models at Scale. *arXiv:2011.05497* [cs.AR]
- [5] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. *arXiv:2005.14165* [cs.CL]
- [6] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. 2021. Synthesizing optimal collective algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 62–75.
- [7] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishu Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Isipir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide and Deep Learning for Recommender Systems. *arXiv:1606.07792* (2016). <http://arxiv.org/abs/1606.07792>
- [8] François Chollet. 2017. Xception: Deep Learning with Depthwise Separable Convolutions. *arXiv:1610.02357* [cs.CV]
- [9] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for YouTube recommendations. In *Proc. 10th ACM Conf. Recommender Systems*. 191–198.
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv:1810.04805* [cs.CL]
- [11] John Duchi, Elad Hazan, and Yoram Singer. 2011. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research* 12, 7 (2011).
- [12] Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Murali Annamaram, Krishnakumar Nair, and Misha Smelyanskiy. 2020. Check-N-Run: A Checkpointing System for Training Recommendation Models. *arXiv:2010.08679* [cs.IR]
- [13] Carlos A. Gomez-Urbe and Neil Hunt. 2016. The Netflix Recommender System: Algorithms, Business Value, and Innovation. *ACM Trans. Manage. Inf. Syst.* 6, 4, Article 13 (Dec. 2016), 19 pages. <https://doi.org/10.1145/2843948>
- [14] Maya R Gupta, Samy Bengio, and Jason Weston. 2014. Training highly multiclass classifiers. *The Journal of Machine Learning Research* 15, 1 (2014), 1461–1492.
- [15] U. Gupta, C. Wu, X. Wang, M. Naumov, B. Reagen, D. Brooks, B. Cottel, K. Hazelwood, M. Hempstead, B. Jia, H. S. Lee, A. Malevich, D. Mudigere, M. Smelyanskiy, L. Xiong, and X. Zhang. 2020. The Architectural Implications of Facebook’s DNN-Based Personalized Recommendation. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 488–501. <https://doi.org/10.1109/HPCA47549.2020.00047>
- [16] Aaron Harlap, Deepak Narayanan, Amar Phanishayee, Vivek Seshadri, Nikhil Devanur, Greg Ganger, and Phil Gibbons. 2018. PipeDream: Fast and Efficient Pipeline Parallel DNN Training. *arXiv:1806.03377* [cs.DC]
- [17] Kim Hazelwood, Sarah Bird, David Brooks, Soumith Chintala, Utku Diril, Dmytro Dzhulgakov, Mohamed Fawzy, Bill Jia, Yangqing Jia, Aditya Kalro, et al. 2018. Applied machine learning at facebook: A datacenter infrastructure perspective. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 620–629.
- [18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. *arXiv:1512.03385* [cs.CV]
- [19] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In *Proc. 26th Int. Conf. World Wide Web*. 173–182.
- [20] Xinran He, Junfeng Pan, Ou Jin, Tianbing Xu, Bo Liu, Tao Xu, Yanxin Shi, Antoine Atallah, Ralf Herbrich, Stuart Bowers, and Joaquin Quiñero Candela. 2014. Practical Lessons from Predicting Clicks on Ads at Facebook. In *Proceedings of the Eighth International Workshop on Data Mining for Online Advertising* (New York, NY, USA) (ADKDD’14).
- [21] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Kurt Keutzer, Ion Stoica, and Joseph E Gonzalez. 2019. Checkmate: Breaking the memory wall with optimal tensor rematerialization. *arXiv preprint arXiv:1910.02653* (2019).
- [22] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2018. Beyond data and model parallelism for deep neural networks. *arXiv preprint arXiv:1807.05358* (2018).
- [23] Biye Jiang, Chao Deng, Huimin Yi, Zelin Hu, Guorui Zhou, Yang Zheng, Sui Huang, Xinyang Guo, Dongyue Wang, Yue Song, et al. 2019. XDL: an industrial

- deep learning framework for high-dimensional sparse data. In *Proceedings of the 1st International Workshop on Deep Learning Practice for High-Dimensional Sparse Data*. 1–9.
- [24] Yimin Jiang, Yibo Zhu, Chang Lan, Bairen Yi, Yong Cui, and Chuanxiong Guo. 2020. A Unified Architecture for Accelerating Distributed DNN Training in Heterogeneous GPU/CPU Clusters. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 463–479. <https://www.usenix.org/conference/osdi20/presentation/jiang>
 - [25] Dhiraj Kalamkar, Evangelos Georganas, Sudarshan Srinivasan, Jianping Chen, Mikhail Shiryayev, and Alexander Heinecke. 2020. Optimizing Deep Learning Recommender Systems Training on CPU Cluster Architectures. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Atlanta, Georgia) (SC '20). IEEE Press, Article 43, 15 pages.
 - [26] Narendra Karmarkar and Richard M. Karp. 1983. *The Differencing Method of Set Partitioning*. Technical Report. USA.
 - [27] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
 - [28] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. 2020. Dynamic tensor rematerialization. *arXiv preprint arXiv:2006.09616* (2020).
 - [29] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 8 (2009), 30–37.
 - [30] ChonLam Lao, Yanfang Le, Kshiteej Mahajan, Yixi Chen, Wenfei Wu, Aditya Akella, and Michael Swift. [n.d.]. ATP: In-network Aggregation for Multi-tenant Learning. ([n.d.]).
 - [31] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668* (2020).
 - [32] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. 2020. PyTorch distributed: experiences on accelerating data parallel training. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3005–3018.
 - [33] Hyeontaek Lim, David G Andersen, and Michael Kaminsky. 2018. 3LC: Lightweight and Effective Traffic Compression for Distributed Machine Learning. *arXiv preprint arXiv:1802.07389* (2018).
 - [34] Yujun Lin, Song Han, Huizi Mao, Yu Wang, and William J Dally. 2017. Deep gradient compression: Reducing the communication bandwidth for distributed training. *arXiv preprint arXiv:1712.01887* (2017).
 - [35] Romain Lopez, Inderjit Dhillon S., and Michael Jordan I. 2021. Learning from eXtreme Bandit Feedback. In *Proc. Association for the Advancement of Artificial Intelligence*.
 - [36] Liang Luo, Peter West, Arvind Krishnamurthy, Luis Ceze, and Jacob Nelson. 2020. PLink: Discovering and Exploiting Datacenter Network Locality for Efficient Cloud-based Distributed Training.
 - [37] Yifei Ma, Balakrishnan (Murali) Narayanaswamy, Haibin Lin, and Hao Ding. 2020. Temporal-Contextual Recommendation in Real-Time (KDD '20). Association for Computing Machinery, New York, NY, USA, 2291–2299.
 - [38] Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Mickevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bitorf, David Brooks, Dehao Chen, Debojyoti Dutta, Udit Gupta, Kim Hazelwood, Andrew Hock, Xinyuan Huang, Atsushi Ike, Bill Jia, Daniel Kang, David Kanter, Naveen Kumar, Jeffery Liao, Guokai Ma, Deepak Narayanan, Tayo Oguntobi, Gennady Pekhimenko, Lillian Pentecost, Vijay Janapa Reddi, Taylor Robie, Tom St. John, Tsuguchika Tabaru, Carole-Jean Wu, Lingjie Xu, Masafumi Yamazaki, Cliff Young, and Matei Zaharia. 2020. MLPerf Training Benchmark. *arXiv:1910.01500* [cs.LG]
 - [39] Azalia Mirhoseini, Hieu Pham, Quoc Le, Mohammad Norouzi, Samy Bengio, Benoit Steiner, Yuefeng Zhou, Naveen Kumar, Rasmus Larsen, and Jeff Dean. 2017. Device Placement Optimization with Reinforcement Learning. <https://arxiv.org/abs/1706.04972>
 - [40] Dheevatsa Mudigere and Whitney Zhao. 2019. HW/SW Co-design for future AI platforms - Large memory unified training platform (Zion). In *2019 OCP Regional Summit, Amsterdam*. <https://2019ocpregionalsummit.sched.com/event/Qyge>
 - [41] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prithvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters. *CoRR* abs/2104.04473 (2021). *arXiv:2104.04473* <https://arxiv.org/abs/2104.04473>
 - [42] Maxim Naumov, John Kim, Dheevatsa Mudigere, Srinivas Sridharan, Xiaodong Wang, Whitney Zhao, Serhat Yilmaz, Changkyu Kim, Hector Yuen, Mustafa Ozdal, Krishnakumar Nair, Isabel Gao, Bor-Yiing Su, Jiyan Yang, and Mikhail Smelyanskiy. 2020. Deep Learning Training in Facebook Data Centers: Design of Scale-up and Scale-out Systems. *arXiv:2003.09518* [cs.DC]
 - [43] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmytro Dzhulgakov, Andrey Malleevich, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. 2019. Deep Learning Recommendation Model for Personalization and Recommendation Systems. *CoRR* abs/1906.00091 (2019). <https://arxiv.org/abs/1906.00091>
 - [44] NVIDIA. 2021. Unified Memory in CUDA 6. <https://developer.nvidia.com/blog/unified-memory-in-cuda-6/>. Accessed: 2021-03-31.
 - [45] OpenAI. 2018. AI and Compute. <https://openai.com/blog/ai-and-compute/>.
 - [46] Satadru Pan, Theano Stavrinou, Yunqiao Zhang, Atul Sikaria, Pavel Zakharov, Abhinav Sharma, Shiva Shankar P, Mike Shuey, Richard Wareing, Monika Gangapuram, Guanglei Cao, Christian Preseau, Pratap Singh, Kestutis Patiejunas, JR Tipton, Ethan Katz-Bassett, and Wyatt Lloyd. 2021. Facebook's Tectonic Filesystem: Efficiency from Exascale. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, 217–231. <https://www.usenix.org/conference/fast21/presentation/pan>
 - [47] Jongsoo Park, Maxim Naumov, Protonu Basu, Summer Deng, Aravind Kalaiah, Daya Khudia, James Law, Parth Malani, Andrey Malevich, Satish Nadathur, Juan Pino, Martin Schatz, Alexander Sidorov, Viswanath Sivakumar, Andrew Tulloch, Xiaodong Wang, Yiming Wu, Hector Yuen, Utku Diril, Dmytro Dzhulgakov, Kim Hazelwood, Bill Jia, Yangqing Jia, Lin Qiao, Vijay Rao, Nadav Rotem, Sungjoo Yoo, and Mikhail Smelyanskiy. 2018. Deep Learning Inference in Facebook Data Centers: Characterization, Performance Optimizations and Hardware Implications. *arXiv:1811.09886* [cs.LG]
 - [48] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems*. 8026–8037.
 - [49] Yanghua Peng, Yibo Zhu, Yangrui Chen, Yixin Bao, Bairen Yi, Chang Lan, Chuan Wu, and Chuanxiong Guo. 2019. A generic communication scheduler for distributed dnn training acceleration. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 16–29.
 - [50] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.
 - [51] Benjamin Recht, Christopher Re, Stephen Wright, and Feng Niu. 2011. Hogwild!: A lock-free approach to parallelizing stochastic gradient descent. *Advances in neural information processing systems* 24 (2011), 693–701.
 - [52] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J. S. Gardner, I. Hubara, S. Idgunji, T. B. Jablin, J. Jiao, T. S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Mickevicius, C. Osborne, G. Pekhimenko, A. T. R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou. 2020. MLPerf Inference Benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 446–459.
 - [53] Amedeo Sapia, Marco Canini, Chen-Yu Ho, Jacob Nelson, Panos Kalnis, Changhoon Kim, Arvind Krishnamurthy, Masoud Moshref, Dan RK Ports, and Peter Richtárik. 2019. Scaling distributed machine learning with in-network aggregation. *arXiv preprint arXiv:1903.06701* (2019).
 - [54] Alexander Sergeev and Mike Del Balso. 2018. Horovod: fast and easy distributed deep learning in TensorFlow. *arXiv preprint arXiv:1802.05799* (2018).
 - [55] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharmarajan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. 2017. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. *arXiv:1712.01815* [cs.AI]
 - [56] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George Drissi, Thore Graepel, and Demis Hassabis. 2017. Mastering the game of Go without human knowledge. *Nature* 550 (10 2017), 354–359. <https://doi.org/10.1038/nature24270>
 - [57] M. Smelyanskiy. 2019. Zion: Facebook Next-Generation Large Memory Training Platform. In *2019 IEEE Hot Chips 31 Symposium (HCS)*. <https://doi.org/10.1109/HOTCHIPS.2019.8875650>
 - [58] Brent Smith and Greg Linden. 2017. Two Decades of Recommender Systems at Amazon.Com. *IEEE Internet Computing* 21, 3 (May 2017), 12–18. <https://doi.org/10.1109/MIC.2017.72>
 - [59] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2014. Going Deeper with Convolutions. *arXiv:1409.4842* [cs.CV]
 - [60] Jakub Tarnawski, Amar Phanishayee, Nikhil R Devanur, Divya Mahajan, and Fanny Nina Paravecino. 2020. Efficient algorithms for device placement of dnn graph operators. *arXiv preprint arXiv:2006.16423* (2020).

- [61] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. *arXiv:1706.03762* [cs.CL]
- [62] M. Xie, K. Ren, Y. Lu, G. Yang, Q. Xu, B. Wu, J. Lin, H. Ao, W. Xu, and J. Shu. 2020. Kraken: Memory-Efficient Continual Learning for Large-Scale Real-Time Recommendations. In *2020 SC20: International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE Computer Society, Los Alamitos, CA, USA, 1–17. <https://doi.org/10.1109/SC41405.2020.00025>
- [63] Jie Amy Yang, Jianyu Huang, Jongsoo Park, Ping Tak Peter Tang, and Andrew Tulloch. 2020. Mixed-Precision Embedding Using a Cache. <https://doi.org/10.48550/ARXIV.2010.11305>
- [64] Jie Amy Yang, Jianyu Huang, Jongsoo Park, Ping Tak Peter Tang, and Andrew Tulloch. 2020. Mixed-Precision Embedding Using a Cache. *arXiv:2010.11305* [cs.LG]
- [65] Jie Amy Yang, Jongsoo Park, Srinivas Sridharan, and Ping Tak Peter Tang. 2020. Training Deep Learning Recommendation Model with Quantized Collective Communications. (2020).
- [66] Yang You, Jing Li, Sashank Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, Kurt Keutzer, and Cho-Jui Hsieh. 2019. Large batch optimization for deep learning: Training bert in 76 minutes. *arXiv preprint arXiv:1904.00962* (2019).
- [67] Jian Zhang, Jiyan Yang, and Hector Yuen. 2018. Training with low-precision embedding tables. In *Systems for Machine Learning Workshop at NeurIPS*, Vol. 2018.
- [68] Sixin Zhang, Anna E Choromanska, and Yann LeCun. 2015. Deep learning with elastic averaging SGD. *Advances in neural information processing systems* 28 (2015), 685–693.
- [69] Whiteny Zhao, Dheevatsa Mudigere, Xiaodong Wang, Jongsoo Park, John Kim, and Mikhail Smelyanskiy. 2019. Accelerator Fabric in Facebook Zion Training System. In *2019 IEEE/ACM International Symposium on Networks-on-Chip (NOCS)*.
- [70] Weijie Zhao, Jingyuan Zhang, Deping Xie, Yulei Qian, Ronglai Jia, and Ping Li. 2019. AIBox: CTR prediction model training on a single node. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. 319–328.
- [71] Qinqing Zheng, Bor-Yiing Su, Jiyan Yang, Alisson Azzolini, Qiang Wu, Ou Jin, Shri Karandikar, Hagay Lupesko, Liang Xiong, and Eric Zhou. 2020. ShadowSync: Performing Synchronization in the Background for Highly Scalable Distributed Training. *CoRR* 2003.03477 (2020).

APPENDIX-A

Compute Benchmarks

We collected and developed a set of operator-level benchmarks which we have also open sourced as part of *PARAM bench*[§], to evaluate the representative problem sizes and shapes on the candidate hardware platforms and to better understand the throughput and latency in compute, memory, and communications.

GEMM benchmark. This benchmark calls cuBLAS GemmEx routine to compute matrix multiplications on configurable problem sizes with multiple precision choices. On the V100 GPU, this benchmark supports FP32 GEMM on the CUDA core and FP16 mixed-precision GEMM on Tensor Core. On the A100 GPU, it additionally supports TF32 GEMM and BF16 GEMM on the Tensor Core.

The benchmark results are shown in Figures 12.

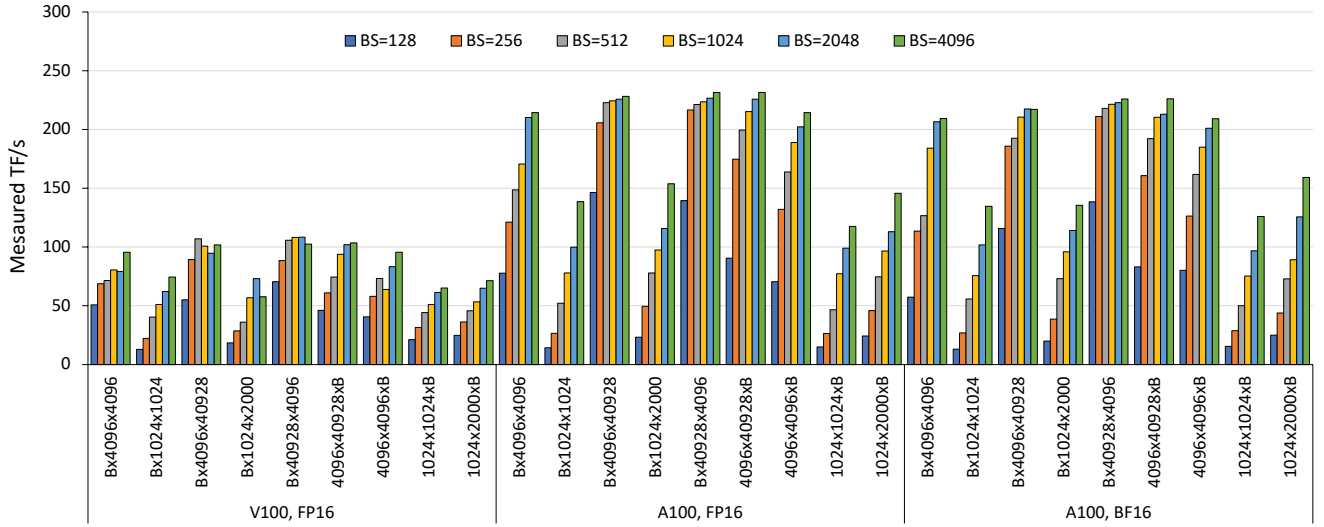


Figure 12: GEMM performance (TF/s) for V100 FP16 vs. A100 FP16/BF16.

MLP benchmark. This benchmark implements the following multilayer perceptron (MLP) layers:

- Batch size = 128, 256, 512, 1024, 2048, 4096;
- 20 MLP layers, where each layer is $1K \times 1K$, $2K \times 2K$ and $4K \times 4K$;
- Each layer has ReLU and final layers has SoftMax;
- Both backward and forward passes, including SGD update as the optimizer after the backward pass;
- Precision support: FP16, BF16, TF32, FP32.

The batch size, layer dimension, and number of layers can be configured to the customized number. We implemented this MLP benchmark using C++, directly implementing FC and FCGradients in the MLP layer using cuBLAS SGEMM/GemmEx function, ReLU with cudNN cudnnActivationForward/ cudnnActivationBackward function, SoftMax with cudnnSoftmaxForward in the forward a customized CUDA kernel for the backward pass, and SGD optimizer with cuBLAS axpy function. This benchmark can be used to project the performance of V100/A100 GPUs using a minimal MLP network without the framework overhead in PyTorch. The benchmark results are shown in Figures 13 and 14.

Memory Benchmark

This benchmark evaluates the achieved memory bandwidth of the embedding kernels described in Section 5. To eliminate the L2 cache effects, a random tensor with 40 MB data (A100 L2 cache size) is allocated to flush the cache.

- Support the evaluation forward and backward pass (the backward pass is fused with optimizer);
- Precision Support: FP32 and FP16;
- Number of rows: 1000000, Number of tables: 64, Embedding dimension: 128, Pooling size: 32, rows per thread block: 32.

The benchmark results are shown in Figures 15 and 16.

[§]<https://github.com/facebookresearch/param>

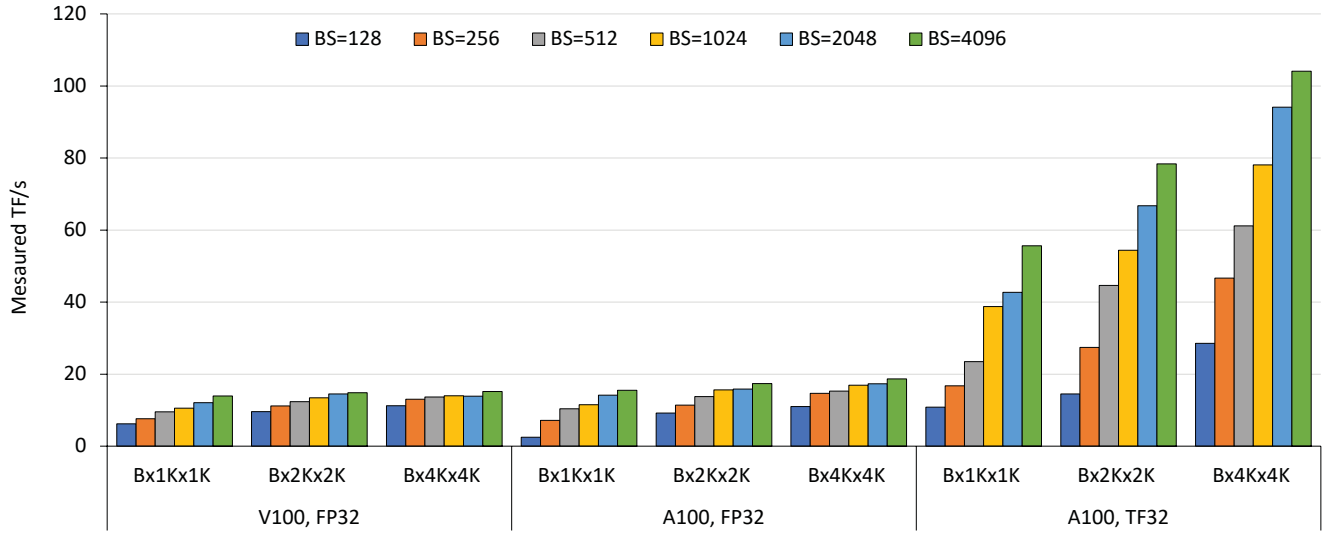


Figure 13: MLP performance for V100 FP32 vs. A100 FP32/TF32.

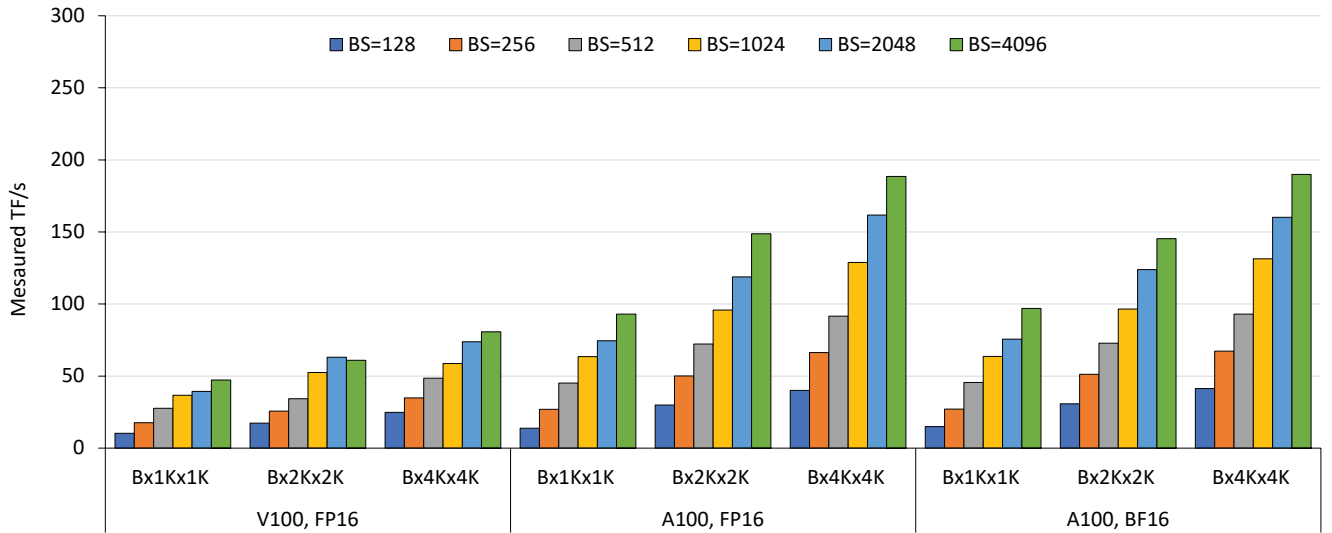


Figure 14: MLP performance for V100 FP16 vs. A100 FP16/BF16.

Communications Benchmark

Low-level collective communication benchmarks, e.g. NVIDIA's NCCL tests or OSU MPI benchmarks, have the following limitations:

- Do not capture the behavior of actual workloads, i.e. exact message sizes, sequence of collective operations, etc. Instead these benchmarks support power-of-two message sizes - helpful to detect network trends.
- Limited to one specific communication library. As the name suggests, NCCL tests works only with NCCL and OSU MPI benchmarks is limited to MPI.

The PARAM comms benchmarks addresses these gaps by:

- Creating common abstractions across platforms (e.g. NVIDIA GPUs, x86 CPUs, Google TPU etc.) to help standardize the benchmarking logic.
- Using PyTorch Process Group APIs to provide a portable interface across different communication libraries (e.g. NCCL, MPI, and UCC).

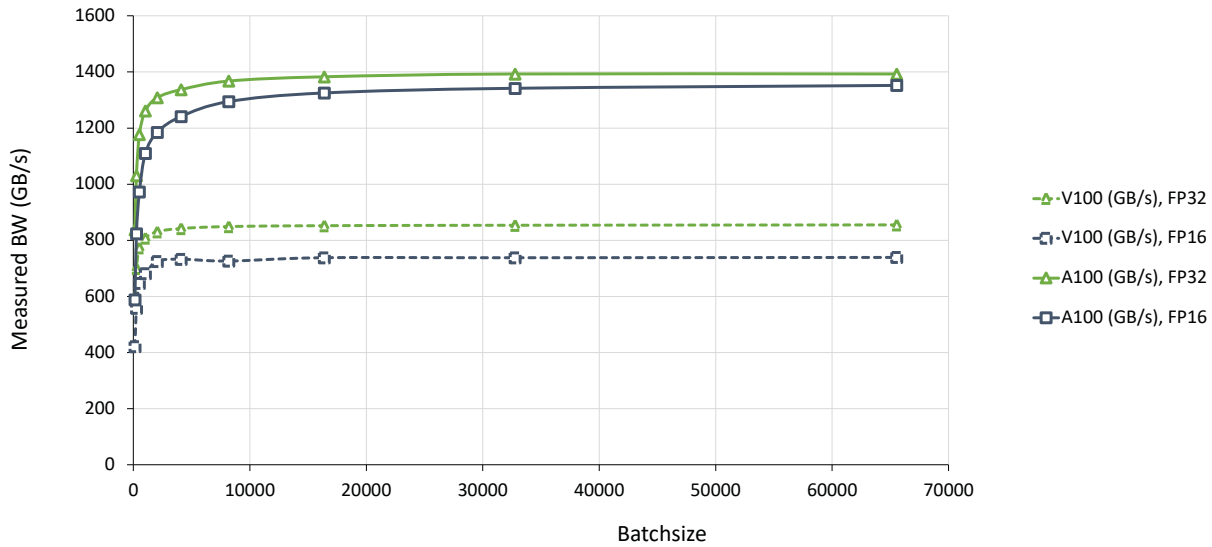


Figure 15: Achieved embedding lookup forward bandwidth using FP32 vs. FP16 on V100 vs. A100.

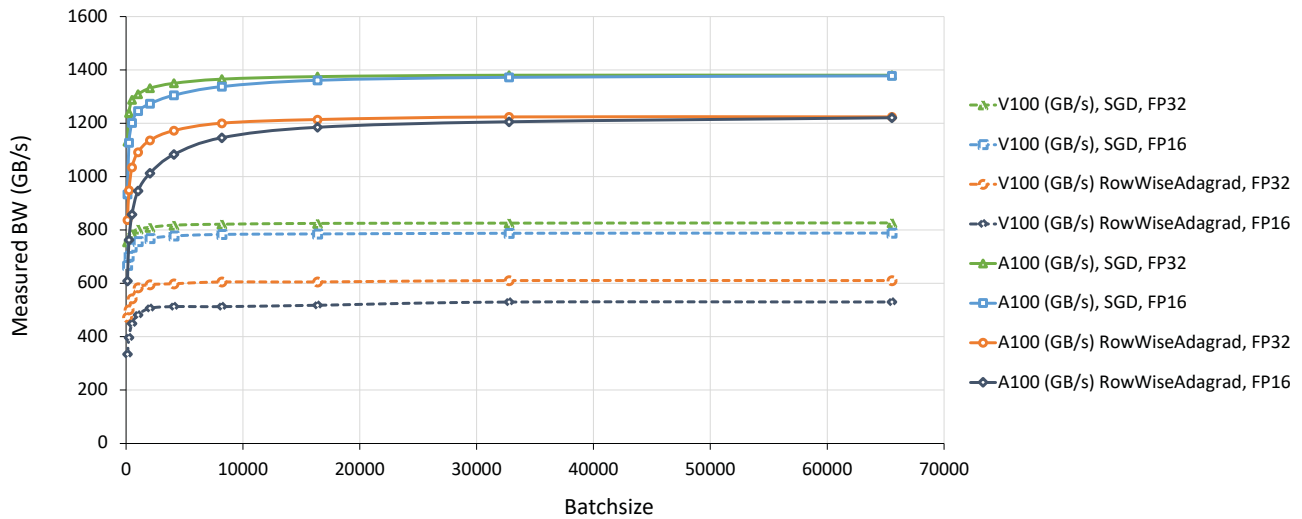


Figure 16: Achieved embedding lookup backward+optimizer bandwidth using FP32 vs. FP16 on V100 vs. A100.

PARAM comms benchmarks supports two types of collective benchmarks:

- Bench mode: Simplest mode of operation similar to NCCL tests. Run single collective in blocking or non-blocking manner across fixed set of message sizes (e.g. power of 2 message sizes). This is mainly used for low-level HW testing
- Replay mode: Replays a trace of collective communication calls to mimic exact workload behavior in terms of collective sizes.

Figure 17 presents AlltoAll and AllReduce benchmark scaling for power-of-two message sizes on 128 GPUs. AlltoAll achieves 7GB/s and is primarily limited by scale-out bandwidth (12.5 GB/s peak; 10.5 GB/s achievable on V100). AllReduce achieves higher bandwidth since it uses NVLINK more effectively.

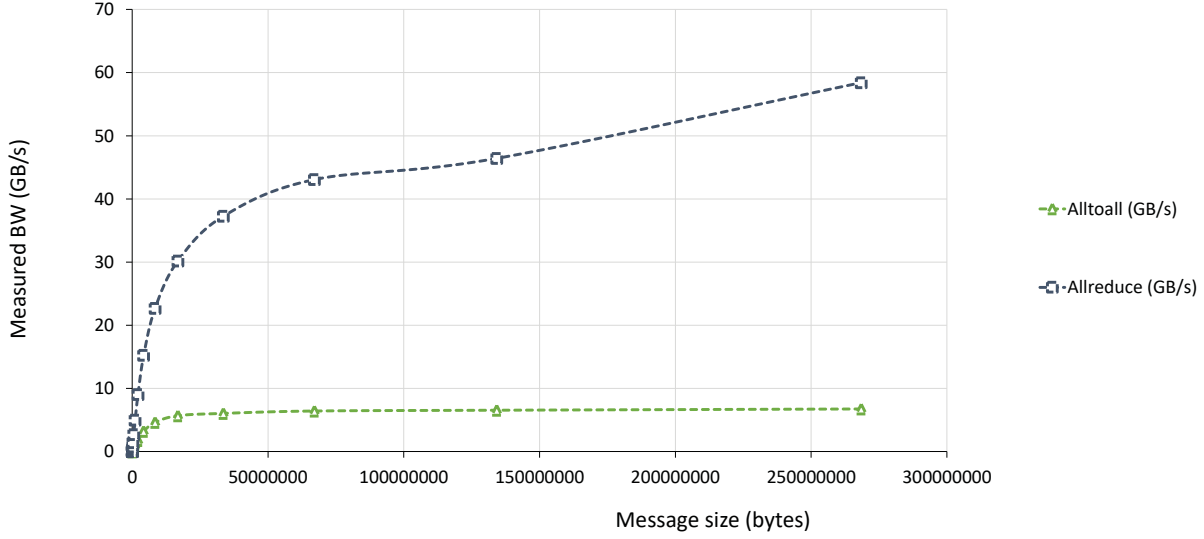


Figure 17: Achieved AlltoAll and AllReduce bandwidth at 128 GPUs

APPENDIX-B

Performance roofline and benchmarking

In order to identify performance gaps, to see how far we are from fully utilizing the platform capabilities - we establish the upper bound for achievable performance using an analytical roofline model. DL-RMs can be broken down into the following major components - 1) bottom MLP; 2) embedding lookup and update; 3) AlltoAll communication of the model-parallel pooled embeddings; 4) interaction and Top MLP; 5) AllReduce communication for the data-parallel MLP gradient synchronization. The execution dependency between these different components are outlined in Fig.18. As discussed above, individually each of these have different characteristics. The latency/performance for each component is dependent on different parts of the system, for instance the embedding ops performance depends on the achievable HBM bandwidth, whereas the MLP performance is bounded by achievable compute flops. Even between the two collective communication primitives - AllReduce performance depends on both the scale-out and scale-up bandwidths, whereas the AlltoAll performance primarily depends on the scale-out bandwidth. With estimates for latencies for these individual components, the overall per-iteration latency can be estimated as shown in Eq. 1

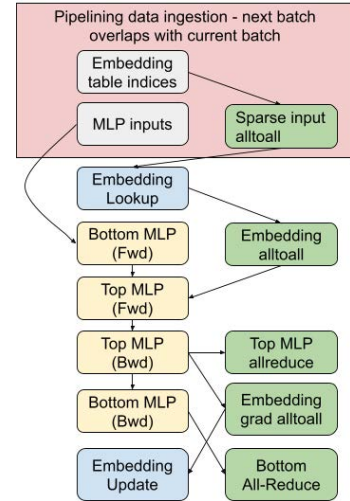


Figure 18: DLRM dependency graph

$$\begin{aligned}
 T_{fwd} &= \max[BotMLP_{fwd}, (Embedding_lookup + alltoall_{fwd})] + Interaction_{fwd} + TopMLP_{fwd} \\
 T_{bwd} &= \max[TopMLP_{bwd} + Interaction_{bwd} + \max\{alltoall_{bwd} + Embedding_update, BotMLP_{bwd}\}, \\
 &\quad (TopMLP_Allreduce + BotMLP_Allreduce)]
 \end{aligned}$$

$$T_{total} = T_{fwd} + T_{bwd} \quad (1)$$

To estimate the performance and latencies for each of these components, we use operator-level benchmarks which allow evaluation of target operator shapes/sizes on candidate HW platforms. We benchmark[¶] the 1) embedding operators, 2) typical MLP sizes, and 3) communication primitives. With these benchmarks we are able to establish the max achievable HBM bandwidth to be 850 GB/s for V100 and 1300 GB/s on A100 GPUs, and for the MLP sizes of interest, achievable compute efficiencies to be up to 78.6% (V100) and 70.5% (A100).

[¶]<https://github.com/facebookresearch/param>

Furthermore, we achieve 7GB/s for 256MB AlltoAll and 60GB/s for 256MB AllReduce. AllReduce is able to achieve higher effective bandwidth since it utilizes both scale-out and NVLINK bandwidths. These benchmarking results and configuration used are detailed in Appendix-A.